

DEC - Dinheiro Em Casa

Projeto Interdisciplinar

Ricardo Nogueira

Nauan Melo

Wellington Picancio

Licenciatura em Engenharia Informática

Sistemas Multimédia II

Professor Helder Pinto

Porto, 2026

Resumo

O presente documento descreve o desenvolvimento do projeto integrador DEC – Dinheiro em Casa, uma plataforma digital de literacia financeira desenvolvida no âmbito da Unidade Curricular de Sistemas Multimédia II da Licenciatura em Engenharia Informática do ISTEC Porto, no ano letivo de 2025-2026.

O sistema foi desenvolvido ao longo de três sprints, seguindo uma metodologia ágil baseada em Scrum. A arquitetura final assenta em dois módulos distintos: uma Plataforma Educativa em .NET Core MVC com base de dados MySQL própria, e um Módulo Financeiro em Django REST com Python integrado e base de dados financeira separada, servidos por Nginx como reverse proxy com TLS. O sistema permite que professores criem turmas e desafios financeiros, e que alunos criem cenários familiares virtuais, registem lançamentos, definam objetivos de poupança e executem simulações financeiras completas com três estratégias distintas, conservadora, moderada e agressiva, comparando resultados em tempo real.

O projeto atingiu todos os objetivos propostos, implementando autenticação com três perfis de acesso, gestão pedagógica completa, motor de cálculo financeiro com 27 testes unitários, backups automáticos diários e pipeline CI/CD no GitHub Actions. As funcionalidades desenvolvidas superam o mínimo exigido, incluindo auto-fill inteligente de cenários, workspace de desafio integrado e comparação paralela das três estratégias de simulação.

Palavras-chave: literacia financeira, simulação financeira, plataforma educativa, microserviços, Docker

Índice

DEC - Dinheiro Em Casa *1*
Resumo *1*
Índice de Figuras *3*
Índice de Tabelas *4*
Lista de Acrónimos e Siglas *5*
1. Introdução *1*
1.1. Enquadramento **1**
1.2. Objetivos **2**
2. Planeamento *3*
2.1. Metodologia **3**
2.2. Recursos **3**
3. Análise e Design *4*
3.1 Personas 4
3.2 Levantamento de Requisitos — FURPS+ 4
3.3 Diagrama de Casos de Uso 7
3.4 Modelo Relacional de Dados 8
3.5 Diagrama de Componentes 9
3.6 Diagrama de Deployment 10
3.7 Glossário Financeiro e Regras CAFIN 11
4. Desenvolvimento *13*
4.1 Funcionalidades do aluno 14
4.2 Funcionalidades do professor 14
4.3 Ferramentas Utilizadas 14
4.3.1 Software de Desenvolvimento 14
4.3.2 Tecnologias e Frameworks 15
4.3.3 Soluções Implementadas 15
4.4 Separação de Bases de Dados 15
4.5 Auto-fill Inteligente de Cenários 15
4.6 Stored Procedures 15
4.7 Backups Automáticos 16
4.8 Resultados 16
4.8.1 Produto Final 16
4.8.2 Testes e Validação 17
4.9 Infraestrutura Docker e Nginx 17
4.10 Padrões de Design Strategy e Factory 19
4.11 Base de Dados - Stored Procedures 20
4.12 Pipeline CI/CD — GitHub Actions 21
4.13 API Django — Endpoints 22
5. Conclusão *23*

5.1. Limitações e Sugestões Futuras..... 23
5.1.1 Dificuldades Encontradas..... 23
5.1.2 Sugestões Futuras..... 23
5.2. Considerações sobre a Inteligência Artificial no Contexto deste Projeto..... 23
Referências:..... 25

Índice de Figuras

Figura 1 - Diagrama de Casos de Uso 8
Figura 2 - Modelo Relacional de Dados 9
Figura 3 - Diagrama de Componentes 10
Figura 4 - Diagrama de Deployment 11
Figura 5 - Dashboard aluno 16
Figura 6 - Desafio do Aluno 16
Figura 7 - Comparação de estratégias 16
Figura 8 - Dashboard do Professor 16
Figura 9 - Postman 17
Figura 10 - Testes Unitários 17
Figura 11 - Docker 18
Figura 12 -Base de dados - SP 20
Figura 13 -Base de dados - SP 20
Figura 14 - Pipeline CI/CD Github 21
Figura 15 - Pipeline CI/CD Github 21

Lista de Acrónimos e Siglas

ISO	International Organization for Standardization
PBL	Project Based Learning
DEC	Dinheiro Em Casa
ISTEC	Instituto Superior de Tecnologias Avançadas do Porto
MVC	Model-View-Controller
API	Application Programming Interface
REST	Representational State Transfer
TLS	Transport Layer Security
HTTPS	Hypertext Transfer Protocol Secure
CI/CD	Continuous Integration / Continuous Deployment
TAN	Taxa Anual Nominal
FURPS	Functionality, Usability, Reliability, Performance, Supportability
DDL	Data Definition Language
ORM	Object-Relational Mapping
EF	Entity Framework

1. Introdução

O presente relatório descreve o desenvolvimento do projeto integrador **DEC – Dinheiro em Casa**, realizado no âmbito da Unidade Curricular de Sistemas Multimédia II, do segundo semestre do segundo ano da Licenciatura em Engenharia Informática (LEI) do ISTEC Porto, no ano letivo de 2025-2026.

O projeto consiste no desenvolvimento de uma plataforma digital de literacia financeira destinada a alunos do 10.º ao 12.º ano, com o objetivo de capacitar jovens para a gestão da economia doméstica através da simulação de cenários reais. A plataforma permite criar famílias virtuais, registar rendimentos e despesas, simular créditos e poupanças, e acompanhar o progresso financeiro através de dashboards interativos.

Este relatório encontra-se organizado em várias secções: enquadramento do projeto, objetivos, planeamento e metodologia, seguido das secções técnicas.

1.1. Enquadramento

O projeto **DEC** surge no contexto do Projeto Integrador da Licenciatura em Engenharia Informática do **ISTEC - Instituto Superior de Tecnologias Avançadas do Porto**, integrando conhecimentos de várias unidades curriculares do segundo semestre do segundo ano, nomeadamente **Tecnologias de Internet III (TINT3)**, **Programação IV (PROG4)**, **Sistemas de Gestão de Bases de Dados (SGBDD)**, **Cálculo Financeiro (CAFIN)** e **Administração de Redes II (ARED2)**.

A equipa de desenvolvimento é composta por três estudantes:
- Ricardo Filipe Barbosa Rocha Nogueira (Team Leader),
- Welington Douglas Picancio Junior
-Nauan Vieira de Melo

Colaboramos num repositório partilhado disponível em:
-https://github.com/nogueira002/dec-dinheiro-em-casa.

1.2. Objetivos

O principal objetivo é desenvolver uma plataforma web de literacia financeira que permita a alunos do ensino secundário simular e compreender a gestão da economia doméstica, sob orientação pedagógica de professores.

Objetivos Específicos:

- Implementar um sistema de autenticação e autorização com quatro perfis de utilizador:
    - Guest
    - Aluno
    - Professor
    - Administrador
- Desenvolver um módulo de gestão financeira que permita criar cenários familiares, registar rendimentos e despesas, e calcular automaticamente a taxa de esforço
- Implementar um microserviço em Python capaz de executar simulações financeiras, incluindo juros compostos, amortização de crédito e projeções a 12/24 meses
- Criar um sistema pedagógico que permita aos professores criar e atribuir desafios financeiros às turmas e acompanhar o progresso dos alunos
- Garantir uma arquitetura distribuída, segura e deployada em containers Docker com comunicação HTTPS

2. Planeamento

2.1. Metodologia

O desenvolvimento do projeto segue uma metodologia ágil baseada em **Scrum**, organizada em três sprints com objetivos e entregas.

Em cada sprint, a equipa procede à análise das user stories disponibilizadas, planeia as tarefas, distribui responsabilidades de forma equitativa e documenta o trabalho realizado. A gestão das tarefas é feita através do **GitHub Projects**, com issues etiquetadas por prioridade e responsável.

O controlo de versões é assegurado pelo **Git**, com o repositório alojado no GitHub, seguindo uma estratégia de branching com branches por funcionalidade e pull requests para integração no ramo principal.

2.2. Recursos

Equipa

- Ricardo Filipe Barbosa Rocha Nogueira — Team Leader, responsável pela coordenação, arquitetura e desenvolvimento
- Welington Douglas Picancio Junior — Desenvolvimento e integração
- Nauan Vieira de Melo — Desenvolvimento e testes

Software

- .NET Core MVC — Portal web (frontend/backend)
- Django REST Framework — API de negócio
- Python — Microserviço de cálculos financeiros
- MySQL — Base de dados relacional
- Docker — Containerização e deploy
- Nginx — Reverse proxy e HTTPS
- draw.io — Modelação UML
- GitHub / GitHub Projects — Controlo de versões e gestão de projeto

Hardware

- Máquinas de desenvolvimento dos elementos da equipa
- Servidor Ubuntu Server para deploy (VM ou equivalente)

3. Análise e Design

3.1 Personas

Persona 1 — Aluno Nome: Jonh Silva, 16 anos, aluno do 11.º ano. Tem pouco conhecimento sobre gestão financeira e utiliza o telemóvel como principal dispositivo. Pretende perceber como gerir o dinheiro da família e poupar para objetivos a curto prazo.

Persona 2 — Professor Nome: Manuela Ferreira, 38 anos, professora de Economia. Utiliza a plataforma para criar desafios pedagógicos, acompanhar o progresso dos alunos e avaliar submissões de forma eficiente.

Persona 3 — Administrador Nome: Paulo Mendes, 45 anos, técnico de sistemas. Responsável pela gestão de utilizadores, monitorização da infraestrutura e garantia da disponibilidade da plataforma.

3.2 Levantamento de Requisitos — FURPS+

F — Functionality (Requisitos Funcionais)

Autenticação e Utilizadores

- F01 — O sistema deve permitir registo de novos utilizadores com email e password
- F02 — O sistema deve enviar email de confirmação após registo
- F03 — O sistema deve permitir recuperação de password
- F04 — O sistema deve suportar 4 perfis de acesso: Guest, Aluno, Professor, Administrador

Gestão Financeira (Aluno)

- F05 — O sistema deve permitir ao aluno criar cenários familiares virtuais
- F06 — O sistema deve permitir registar rendimentos fixos e variáveis
- F07 — O sistema deve permitir registar despesas categorizadas
- F08 — O sistema deve calcular automaticamente a taxa de esforço do agregado
- F09 — O sistema deve permitir definir objetivos de poupança com prazo e valor
- F10 — O sistema deve simular juros compostos em produtos de poupança
- F11 — O sistema deve simular planos de amortização de crédito
- F12 — O sistema deve gerar projeções financeiras a 12 e 24 meses
- F13 — O sistema deve apresentar dashboard com saldos e evolução de poupança

Gestão Pedagógica (Professor)

- F14 — O sistema deve permitir ao professor criar e gerir turmas
- F15 — O sistema deve permitir adicionar e remover alunos de turmas
- F16 — O sistema deve permitir criar desafios financeiros e atribuí-los a turmas
- F17 — O sistema deve permitir avaliar submissões dos alunos
- F18 — O sistema deve permitir visualizar estatísticas de progresso dos alunos

Administração

- F19 — O sistema deve permitir ao administrador gerir utilizadores
- F20 — O sistema deve permitir monitorizar a integridade da infraestrutura

U — Usability (Usabilidade)

- U01 — A interface deve ser responsiva e funcionar em desktop e mobile
- U02 — A navegação deve ser intuitiva, permitindo ao aluno completar tarefas principais sem formação prévia
- U03 — O dashboard deve apresentar informação financeira de forma visual e gráfica
- U04 — O sistema deve apresentar feedback automático sobre cumprimento de metas pedagógicas
- U05 — O layout deve seguir as especificações do Brand Guidelines dado para o projeto (cores, tipografia, logo)

R — Reliability (Fiabilidade)

- R01 — O sistema deve estar disponível 99% do tempo em período letivo
- R02 — Os dados financeiros dos alunos não devem ser perdidos em caso de falha
- R03 — O sistema deve realizar backups diários da base de dados MySQL
- R04 — Em caso de erro no microserviço Python, o portal deve apresentar mensagem de erro clara sem crashar

P — Performance (Desempenho)

- P01 — O portal deve carregar em menos de 3 segundos em condições normais
- P02 — O microserviço Python deve devolver resultados de simulação em menos de 5 segundos
- P03 — O sistema deve suportar pelo menos 30 utilizadores simultâneos (dimensão de turma)
- P04 — As queries à base de dados MySQL não devem exceder 1 segundo de execução

S — Supportability (Suportabilidade)

- S01 — O sistema deve ser deployado em containers Docker para facilitar manutenção

- S02 — O código deve seguir boas práticas e estar documentado para facilitar alterações futuras
- S03 — O sistema deve ter logs de erros e atividade para diagnóstico
- S04 — A arquitetura em microserviços deve permitir atualizar componentes independentemente

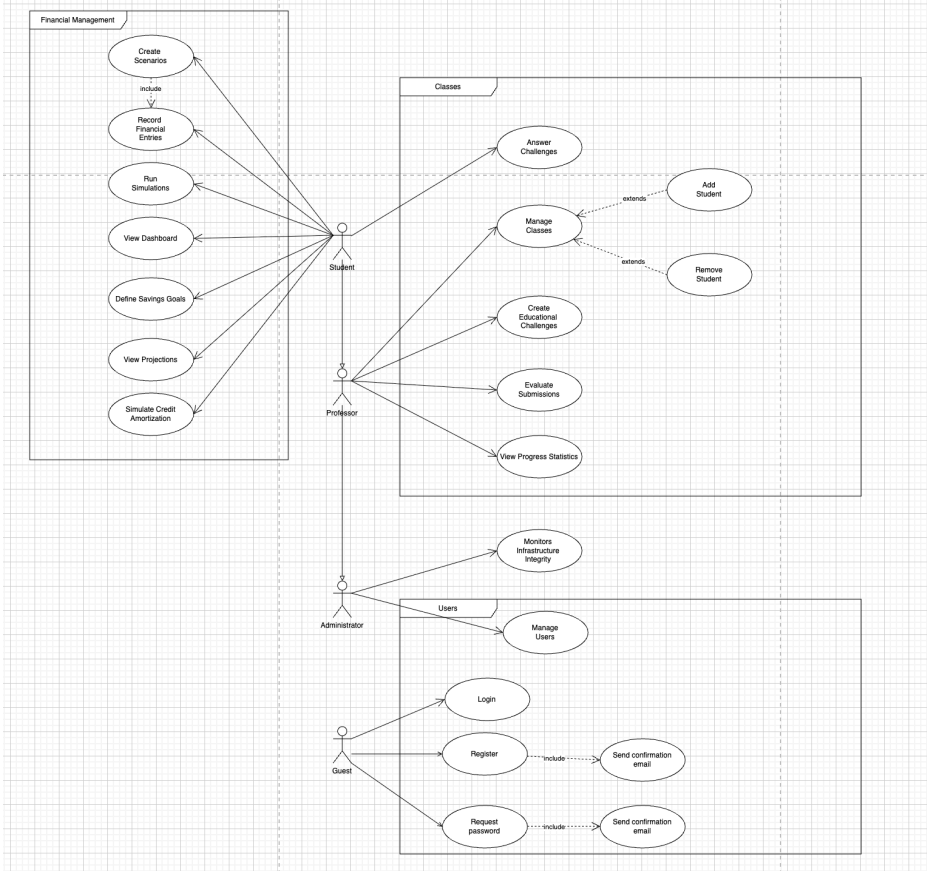
+ — Constraints (Restrições)

- C01 — O portal deve ser desenvolvido em .NET Core MVC
- C02 — A API de negócio deve ser desenvolvida em Django REST
- C03 — O microserviço de cálculos deve ser desenvolvido em Python
- C04 — A base de dados deve ser MySQL
- C05 — O deploy deve ser feito com Docker Compose
- C06 — A comunicação deve ser feita via HTTPS com Nginx como reverse proxy
- C07 — O sistema deve implementar os padrões Strategy e Factory no microserviço Python

3.3 Diagrama de Casos de Uso

O diagrama de casos de uso (versão 1.1) representa as funcionalidades do sistema DEC na perspetiva dos seus quatro atores principais: Guest, Aluno, Professor e Administrador. Existe uma relação de herança entre os atores, o Professor herda todas as funcionalidades do Aluno, e o Administrador herda todas as funcionalidades do Professor.

O sistema está organizado em três subsistemas: **Financial Management** (funcionalidades financeiras do aluno), **Classes** (gestão pedagógica do professor) e **Users** (autenticação e gestão de utilizadores).



3.4 Modelo Relacional de Dados

O modelo relacional de dados foi analisado e atualizado relativamente à versão fornecida inicialmente. As principais alterações introduzidas foram a adição do campo `class_id` na tabela `users`, os campos `location` e `household_size` na tabela `scenarios`, o campo `year` na tabela `entries`, o campo `current_value` na tabela `objectives`, o campo `simulation_type` na tabela `simulationshistory`, e a criação das tabelas `challenges` e `challenge_submissions` para suportar a gestão pedagógica.

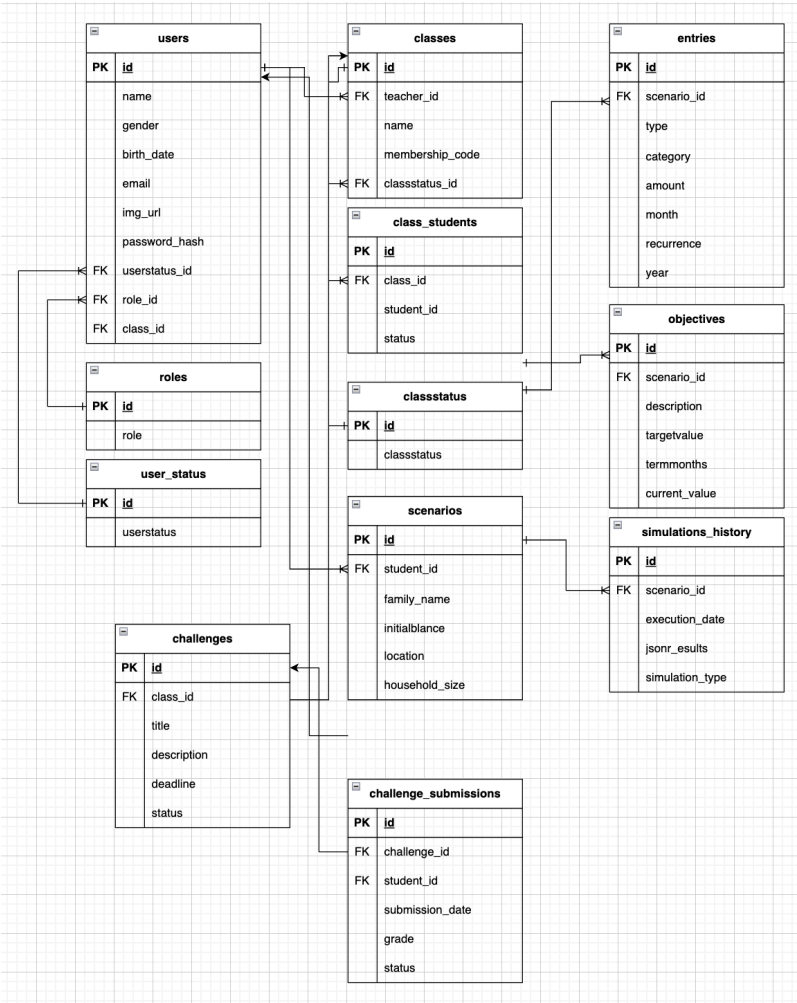


Figura 2 - Modelo Relacional de Dados

3.5 Diagrama de Componentes

O diagrama de componentes representa a arquitetura do sistema e a comunicação entre os seus quatro componentes principais: o Portal .NET Core MVC, a API Django REST, o Microserviço Python de cálculos financeiros e a base de dados MySQL. A comunicação entre o portal e a API é feita via HTTPS/REST, a API comunica com o microserviço Python também via REST, e ambos acedem à base de dados MySQL na rede interna Docker.

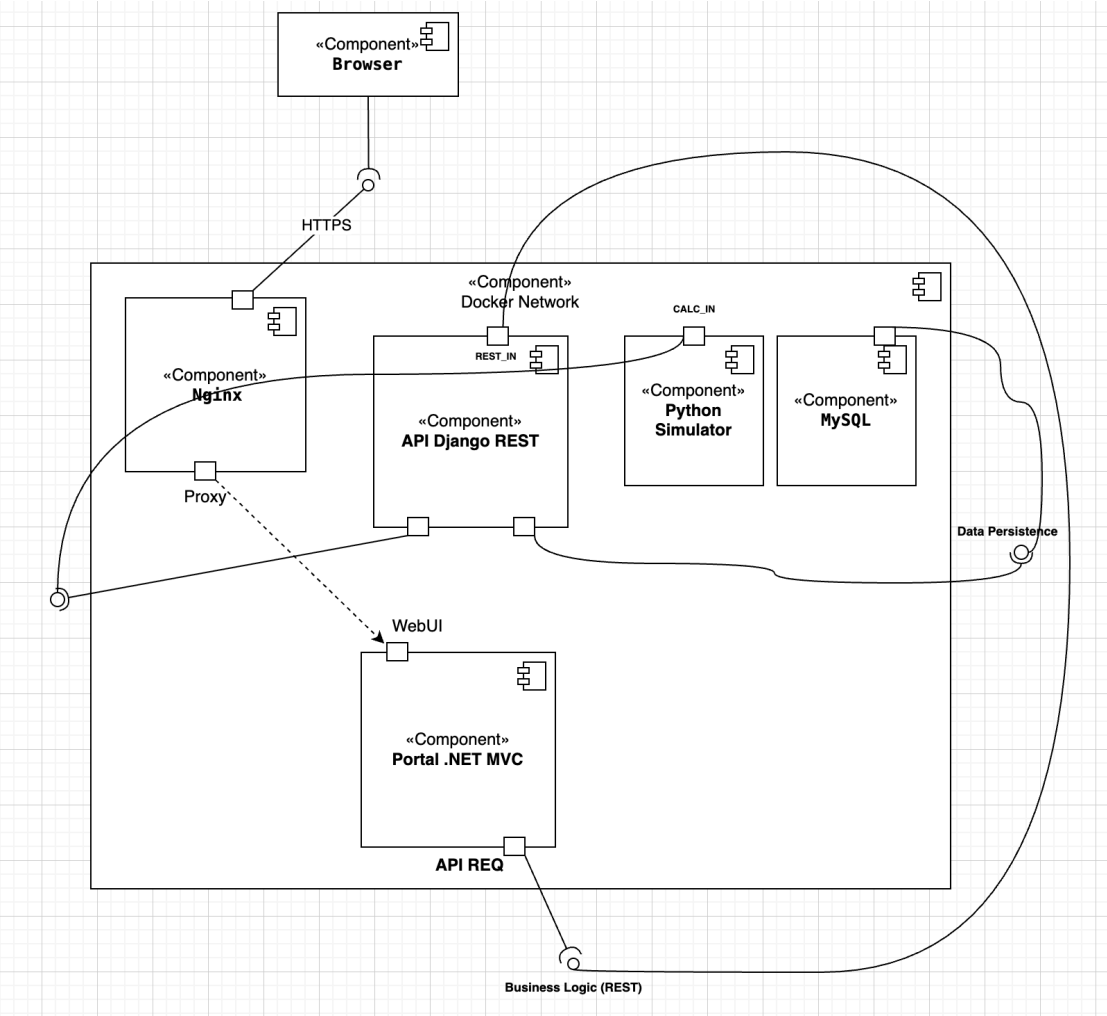


Figura 3 - Diagrama de Componentes

3.6 Diagrama de Deployment

O diagrama de deployment representa a topologia de execução do sistema, com todos os serviços containerizados via Docker Compose numa única VM Ubuntu Server. O Nginx atua como reverse proxy na porta 443, encaminhando os pedidos para o container .NET MVC, que comunica com a API Django e o microserviço Python na rede interna Docker. O MySQL é acessível apenas internamente, com backups diários para o host.

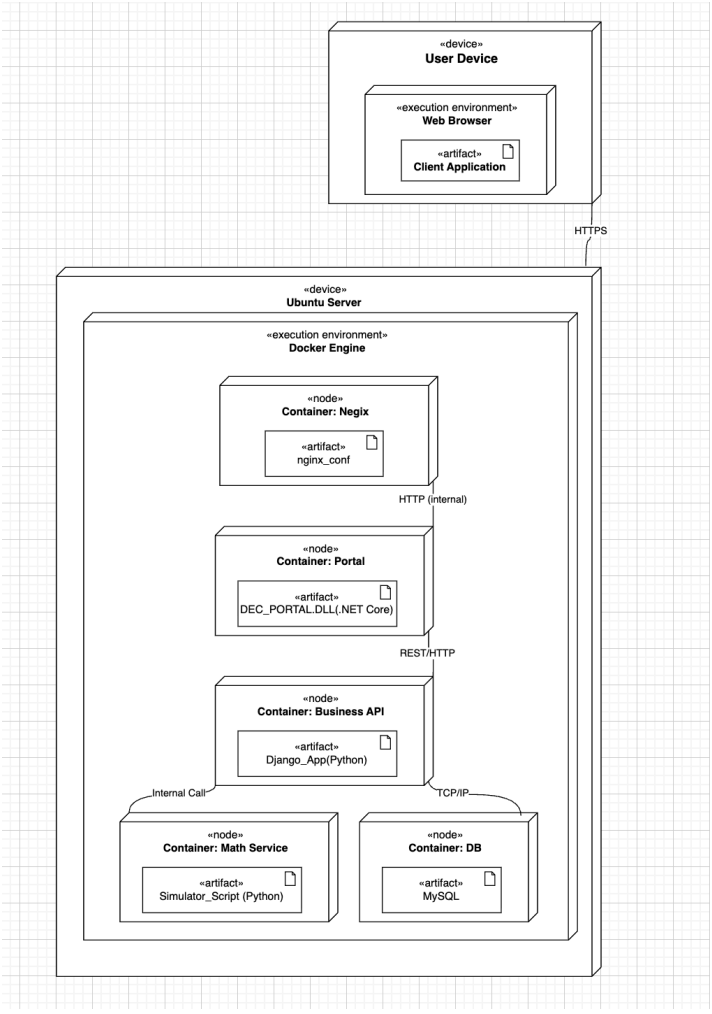


Figura 4 - Diagrama de Deployment

3.7 Glossário Financeiro e Regras CAFIN

O módulo financeiro do sistema DEC implementa os seguintes modelos matemáticos, baseados nas regras da Unidade Curricular de Cálculo Financeiro (CAFIN):

Taxa de Esforço

A taxa de esforço representa a percentagem do rendimento mensal líquido destinada a encargos fixos, medindo a pressão financeira do agregado familiar.

TE = (Encargos Fixos Mensais / Rendimento Líquido Mensal) × 100

Taxa de Esforço	Classificação
TE < 30%	Confortável
30% ≤ TE ≤ 35%	Equilibrado
35% < TE ≤ 50%	Preocupante
TE > 50%	Risco Elevado

Capitalização Composta

Utilizada para projeção de poupanças e investimentos, onde os juros gerados são adicionados ao capital e geram novos juros.

M = C × (1 + i)^n

M = Montante final
C = Capital inicial
i = Taxa de juro mensal (TAN / 12)
n = Número de meses

Sistema de Amortização Francês (Price)

Utilizado para cálculo de prestações de crédito com prestação mensal constante.

R = PV × [i / (1 - (1 + i)^-n)]

R = Prestação mensal constante
PV = Valor do empréstimo
i = Taxa de juro mensal (TAN / 12)
n = Número de prestações

Projeção de Poupança com Inflação

Ajusta as despesas variáveis ao custo de vida futuro, calculando o saldo acumulado ao longo do tempo.

Despesa futura = Despesa atual × (1 + taxa_inflação)^n
Saldo mensal = Rendimentos - Despesas ajustadas
Poupança acumulada = Soma dos saldos mensais ao longo de n meses

Parâmetros por Estratégia de Simulação

As estratégias representam cenários de mercado distintos, não comportamentos da família. Os dados da família são os mesmos o que muda são as taxas aplicadas:

Parâmetro	Conservadora	Moderada	Agressiva
Taxa juro poupança (TAN)	1,5%	3,0%	5,0%
Taxa de inflação anual	4,0%	2,5%	1,5%
Taxa juro crédito (TAN)	6,0%	4,5%	3,0%

4. Desenvolvimento

O desenvolvimento do projeto DEC seguiu a metodologia ágil Scrum, organizada em sprints com objetivos e entregas bem definidos. Durante o Sprint B, a equipa focou-se na criação da estrutura base de todos os componentes do sistema, tendo como principais objetivos a implementação da plataforma educativa em .NET MVC, o módulo financeiro em Django REST e a definição das bases de dados.

Uma das principais decisões tomadas durante este sprint foi a reestruturação da arquitetura do sistema, resultante da adenda às especificações fornecida pelos docentes. A arquitetura evoluiu de três serviços separados (Portal MVC, API Django e Python Simulator) para dois módulos distintos e independentes: a **Plataforma Educativa** (.NET MVC com base de dados própria) e o **Módulo Financeiro** (Django REST API com Python integrado e base de dados própria). Esta decisão implicou a reorganização do trabalho da equipa e a migração do código Python das strategies e factories para dentro da estrutura Django.

No que diz respeito ao módulo financeiro, foram implementadas as classes base dos padrões de design **Strategy** e **Factory**, preparando a estrutura para a implementação completa dos cálculos financeiros no Sprint C. O padrão Strategy foi aplicado para suportar três modelos de simulação — conservador, moderado e agressivo — enquanto o padrão Factory foi utilizado para a criação de cenários familiares distintos, nomeadamente estudante deslocado, casal e família monoparental.

A base de dados financeira dec_financeira foi definida com três tabelas principais: entries, para registo de lançamentos financeiros; objectives, para definição de objetivos de poupança; e simulationshistory, para persistência do histórico de simulações executadas. A separação física das bases de dados, dec_educativa para dados educativos e dec_financeira para dados financeiros, garante a independência entre os dois módulos do sistema.

Reestruturação da Arquitetura

Na sequência da adenda às especificações, a equipa procedeu à reestruturação da arquitetura do sistema. O microserviço Python que existia como serviço independente foi integrado dentro do projeto Django, eliminando um container Docker e simplificando a comunicação entre componentes. Esta solução permitiu manter todo o código Python de cálculos financeiros organizado dentro do módulo Django, sem perda de trabalho já realizado.

O Sprint C focou-se na implementação completa das funcionalidades end-to-end do sistema, integrando o Portal .NET MVC com o Módulo Financeiro Django e validando todas as funcionalidades principais com testes unitários e de integração.

No módulo financeiro, foram implementados os quatro calculators financeiros, taxa de esforço, capitalização composta, amortização pelo método Price e projeção de poupança com inflação, cada um com testes unitários independentes. As três strategies de simulação foram implementadas com os respetivos parâmetros de mercado, e os endpoints REST CRUD foram criados para lançamentos financeiros, objetivos de poupança e histórico de simulações. O endpoint /api/simulate/ foi configurado para guardar automaticamente cada simulação no histórico.

No Portal .NET MVC, foram implementadas todas as funcionalidades.

4.1 Funcionalidades do aluno

- Criação e gestão de cenários familiares;
- Registo de lançamentos financeiros;
- Definição de objetivos de poupança;
- Execução de simulações;
- Auto-fill inteligente e workspace de desafio integrado;

4.2 Funcionalidades do professor

- Criação de turmas;
  Desafios com enunciado Markdown,
- Avaliação de submissões e acompanhamento de progress;

A base de dados educativa foi enriquecida com seis stored procedures para operações críticas de CRUD financeiro.

A infraestrutura foi completada com a configuração de backups automáticos diários às 02:00 com retenção de 7 dias, e a pipeline CI/CD no GitHub Actions com lint, testes Python e build .NET em cada push.

4.3 Ferramentas Utilizadas

4.3.1 Software de Desenvolvimento

- Visual Studio Code e Visual Studio 2022
- PyCharm
- DBeaver
- Postman
- draw.io

4.3.2 Tecnologias e Frameworks

- ASP.NET Core MVC com Entity Framework Core
- Django REST Framework com Python 3.12
- MySQL 8.0: bases de dados dec_educativa e dec_financeira
- Docker e Docker Compose: containerização dos 5 serviços
- Nginx com TLS: reverse proxy e HTTPS na porta 443
- GitHub Actions: pipeline CI/CD automático
- pytest: testes unitários Python

4.3.3 Soluções Implementadas

Padrão Strategy para Simulações Financeiras

O motor de cálculo implementa o padrão Strategy com três classes concretas:

1. **ConservativeStrategy;**
2. **ModerateStrategy;**
3. **AggressiveStrategy;**

Cada uma com parâmetros de mercado distintos. O endpoint /api/simulate/ seleciona dinamicamente a strategy com base no parâmetro recebido, executa todos os cálculos e guarda automaticamente o resultado no histórico de simulações.

4.4 Separação de Bases de Dados

A decisão de separar fisicamente os dados educativos (dec_educativa) dos dados financeiros (dec_financeira) garantiu a independência entre os dois módulos. Assim a consistência entre bases de dados consegue ser assegurada pela lógica aplicacional no Portal MVC, que valida a existência do cenário antes de cada pedido ao módulo financeiro.

4.5 Auto-fill Inteligente de Cenários

Ao selecionar um cenário no formulário de simulação, o sistema agrega automaticamente os lançamentos do aluno, receitas como rendimento mensal, despesas recorrentes como encargos fixos e despesas não recorrentes como despesas variáveis, preenchendo os campos sem intervenção manual.

4.6 Stored Procedures

Foram implementadas seis stored procedures na base de dados educativa para operações críticas de CRUD financeiro, seguindo o padrão com parâmetros de saída p_ha_erro e p_mensagem para tratamento de erros a nível de base de dados.

4.7 Backups Automáticos

Os scripts backup.sh e restore.sh foram configurados como cron job diário às 02:00, com retenção automática de 7 dias e compressão gzip, garantindo recuperação de dados em caso de falha.

4.8 Resultados

4.8.1 Produto Final

O sistema DEC – Dinheiro em Casa foi entregue como uma plataforma web completamente funcional, acessível via HTTPS, com três perfis de utilizador distintos e integração completa entre os dois módulos.

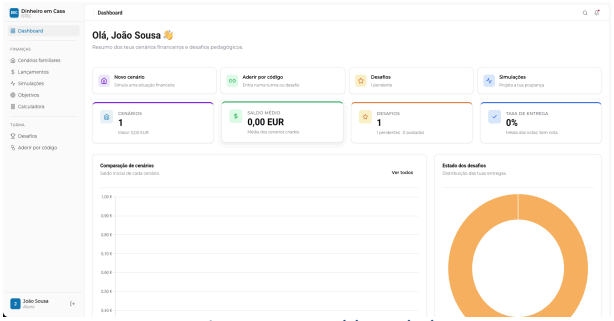


Figura 5 - Dashboard aluno

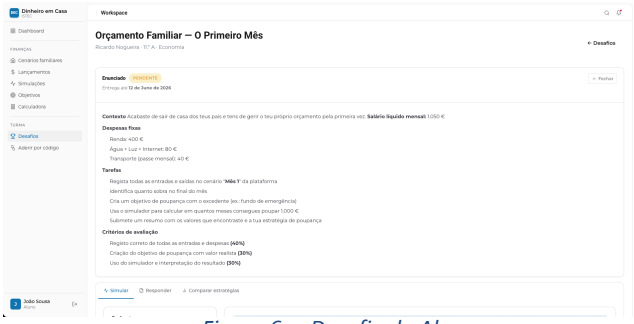


Figura 6 - Desafio do Aluno

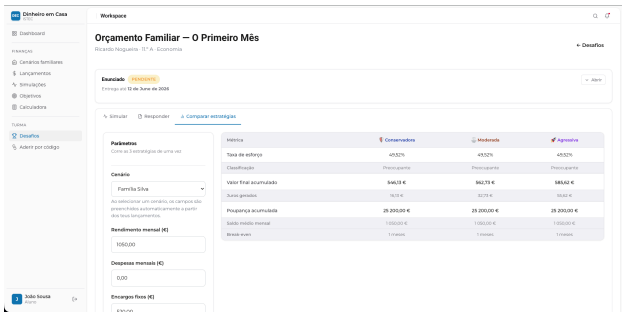


Figura 7 - Comparação de estratégias

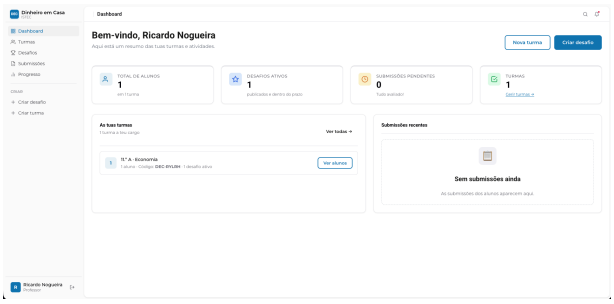


Figura 8 - Dashboard do Professor

4.8.2 Testes e Validação

O módulo financeiro foi validado com 27 testes unitários pytest distribuídos por 7 ficheiros de teste, cobrindo todos os calculators e strategies. A cobertura de testes atingiu mais de 50% do código Python. Foi exportada uma coleção Postman com exemplos de request e response para todos os endpoints da API Django

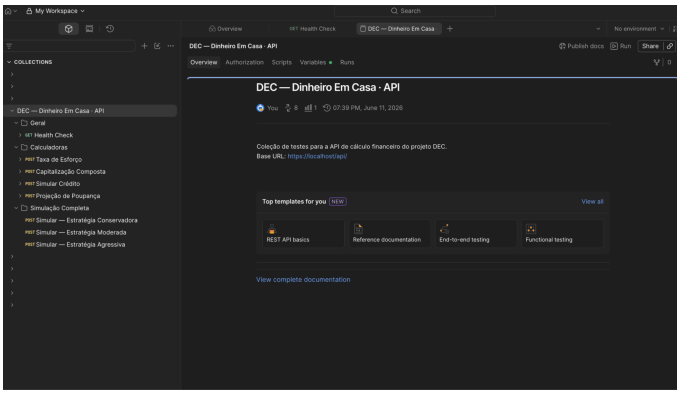


Figura 9 - Postman

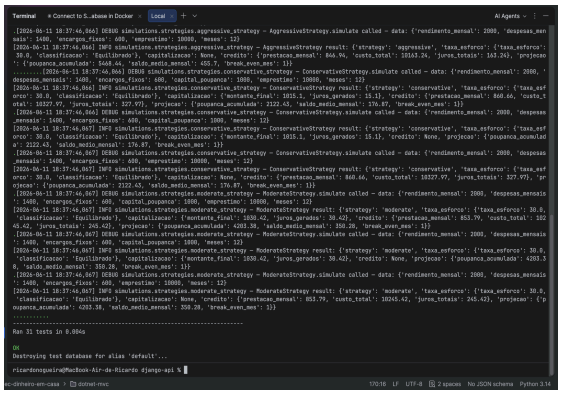


Figura 10 - Testes Unitários

4.9 Infraestrutura Docker e Nginx

Conteinerização com Docker Compose

O sistema é composto por cinco containers Docker orquestrados com Docker Compose, todos comunicando através de uma rede interna privada, garantindo que as bases de dados nunca ficam expostas ao exterior:

Container	Tecnologia	Função	Porta interna
portal	.NET Core MVC	Portal educativo	8080
django-api	Django + Python	Motor de cálculo	8000
mysql	MySQL 8.0	BD educativa dec_educativa	3306
mysql-financeira	MySQL 8.0	BD financeira dec_financeira	3306
nginx	Nginx Alpine	Reverse proxy	80/443

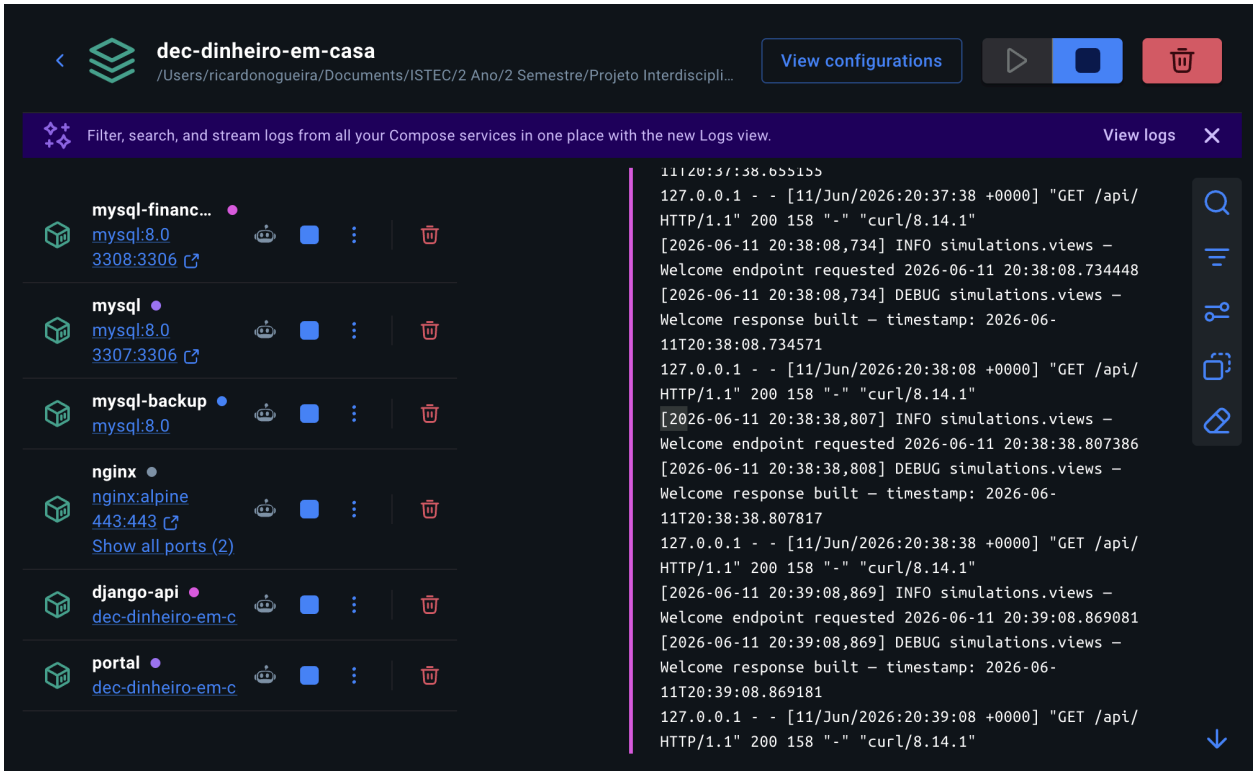


Figura 11 - Docker

O Docker Compose define health checks para todos os containers, garantindo que os serviços dependentes só arrancam após os serviços base estarem disponíveis. O portal depende do MySQL educativo, o Django depende do MySQL financeiro, e o Nginx depende de ambos o portal e o Django estarem saudáveis.

Configuração do Nginx

O Nginx atua como único ponto de entrada do sistema, exposto nas portas 80 e 443. Todo o tráfego HTTP é automaticamente redirecionado para HTTPS. O routing é feito por prefixo de URL:

- Pedidos para /api/ são encaminhados para o container django-api na porta 8000
- Todos os outros pedidos são encaminhados para o container portal na porta 8080

Esta separação garante que o portal MVC e a API Django são acessíveis através de um único domínio e certificado TLS, sem exposição direta dos serviços internos.

Segurança da Infraestrutura

A segurança da infraestrutura assenta em várias camadas:

- As bases de dados MySQL nunca são expostas externamente, apenas acessíveis dentro da rede Docker interna
- O Nginx implementa TLS 1.2 e 1.3 com cifras seguras
- O portal MVC utiliza anti-forgery tokens em todos os formulários POST
- A sessão é gerida por cookie com validação em cada pedido
- Os containers correm com utilizadores não-root sempre que possível

Topologia de Rede Docker

Todos os containers comunicam através de uma rede bridge interna chamada internal, criada pelo Docker Compose. Esta rede isola completamente os serviços do exterior — apenas o Nginx tem portas expostas ao host.

As bases de dados MySQL nunca têm portas expostas ao exterior em ambiente de produção. Em ambiente de desenvolvimento local, as portas 3307 e 3308 são opcionalmente expostas para acesso via ferramentas como DBeaver.

Configuração TLS no Nginx

O Nginx está configurado com TLS 1.2 e 1.3, rejeitando versões mais antigas consideradas inseguras. A configuração inclui:

- Redirecionamento automático de HTTP (porta 80) para HTTPS (porta 443)
- Certificado TLS configurado via variáveis de ambiente
- Cifras seguras com **HIGH:!aNULL:!MD5**
- Headers de segurança: **X-Real-IP**, **X-Forwarded-For**, **X-Forwarded-Proto**

Segurança Aplicacional

O portal MVC implementa várias camadas de segurança aplicacional:

- **Anti-forgery tokens** - todos os formulários POST incluem um token CSRF gerado pelo servidor, prevenindo ataques Cross-Site Request Forgery
- **Sessão por cookie** - a sessão do utilizador é gerida por cookie HttpOnly com validação em cada pedido
- **Autorização por role** - o acesso a cada área é protegido por atributos de autorização que verificam o role do utilizador na sessão
- **Validação de input** - todos os formulários têm validação server-side com Data Annotations do .NET

Backups e Recuperação

Os backups são configurados como cron job dentro do container de backup, executando diariamente às 02:00. O processo:

- Executa mysqldump em ambas as bases de dados
- Comprime o resultado com gzip
- Guarda no volume de backups com timestamp
- Remove backups com mais de 7 dias automaticamente

O script **restore.sh** permite recuperar qualquer backup disponível, especificando a data pretendida como argumento.

Health Checks

Todos os containers têm health checks configurados no Docker Compose:

Container	Health Check	Intervalo
portal	curl http://localhost:8080/	30s
django-api	curl http://localhost:8000/api/	30s
mysql	mysqladmin ping	10s
mysql-financeira	mysqladmin ping	10s
nginx	wget http://localhost/	30s

Os health checks garantem que o Nginx só começa a receber tráfego após o portal e o Django estarem completamente disponíveis, e que o portal e o Django só arrancam após as respetivas bases de dados estarem prontas.

4.10 Padrões de Design Strategy e Factory

Padrão Strategy

O padrão Strategy foi implementado no módulo financeiro Django para permitir a seleção dinâmica do modelo de simulação em tempo de execução. A classe abstrata **BaseStrategy**

define o contrato que todas as estratégias devem respeitar, através do método:
simulate(data: dict) -> dict

BaseStrategy (ABC)

- **ConservativeStrategy** -> TAXA_JURO_POUPANCA=1.5%, TAXA_INFLACAO=4.0%
- **ModerateStrategy** -> TAXA_JURO_POUPANCA=3.0%, TAXA_INFLACAO=2.5%
- **AggressiveStrategy** -> TAXA_JURO_POUPANCA=5.0%, TAXA_INFLACAO=1.5%

Cada strategy recebe os dados financeiros do cenário, aplica os seus parâmetros de mercado e chama os quatro calculators, taxa de esforço, capitalização composta, amortização Price e projeção de poupança.
O endpoint /api/simulate/ seleciona a strategy correta com base no parâmetro strategy recebido no pedido.

Padrão Factory (Scenarios)
O padrão Factory foi implementado para encapsular a criação de cenários familiares com perfis pré-definidos. A classe abstrata **BaseScenario** define o contrato através do método ***create_scenario(data: dict) -> dict.***

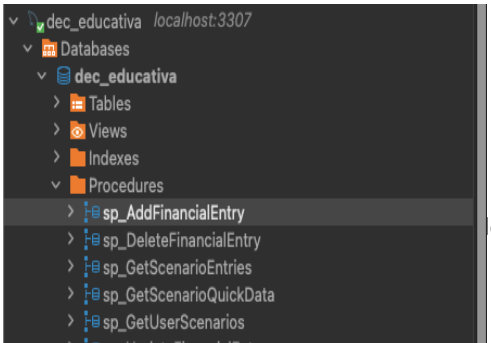
BaseScenario (ABC)

- StudentScenario → Estudante deslocado, rendimento baixo
- SingleParentScenario → Família monoparental, um rendimento
- CoupleScenario → Casal, dois rendimentos partilhados

A utilização de classes abstratas Python (ABC) garante que qualquer nova estratégia ou cenário adicionado no futuro é obrigado a implementar os métodos definidos, prevenindo erros em tempo de desenvolvimento.

4.11 Base de Dados - Stored Procedures

A base de dados educativa **dec_educativa** e composta por seis stored procedures para operações críticas de CRUD financeiro, seguindo o padrão com parâmetros de saída p_ha_erro e p_mensagem para tratamento de erros a nível de base de dados. Este padrão permite que o portal MVC receba mensagens de erro descritivas sem expor detalhes internos da base de dados.



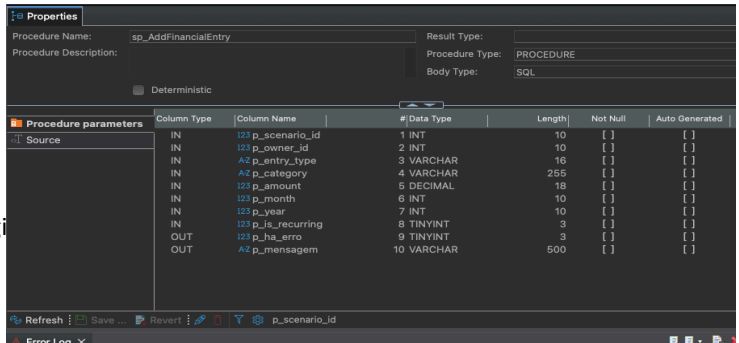


Figura 12 - Base de dados - SP

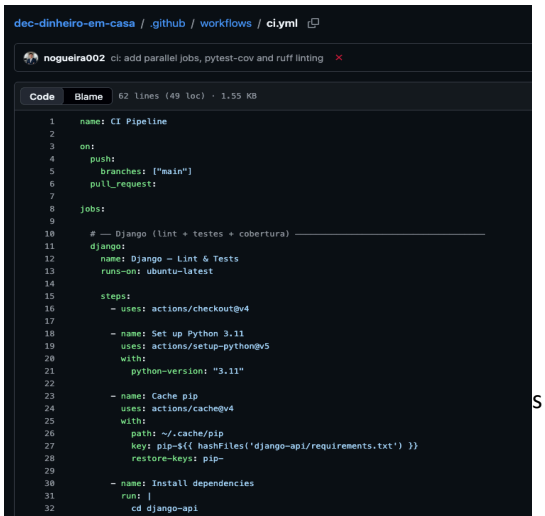
Figura 13 - Base de dados - SP

Stored Procedure	Descrição
sp_GetUserScenarios	Lista todos os cenários de um utilizador
sp_GetScenarioEntries	Lista todos os lançamentos de um cenário
sp_AddFinancialEntry	Adiciona um lançamento financeiro com validação
sp_UpdateFinancialEntry	Atualiza um lançamento existente com validação
sp_DeleteFinancialEntry	Remove um lançamento com validação
sp_GetScenarioQuickData	Agrega dados rápidos do cenário para o dashboard

Todas as stored procedures que modificam dados recebem parâmetros de saída OUT p_ha_erro TINYINT e OUT p_mensagem VARCHAR(255), permitindo ao portal MVC verificar o resultado da operação e apresentar mensagens adequadas ao utilizador.

4.12 Pipeline CI/CD — GitHub Actions

O repositório GitHub está configurado com uma pipeline de integração contínua que é executada automaticamente em cada push para qualquer branch. A pipeline garante que nenhum código com erros é integrado no ramo principal.



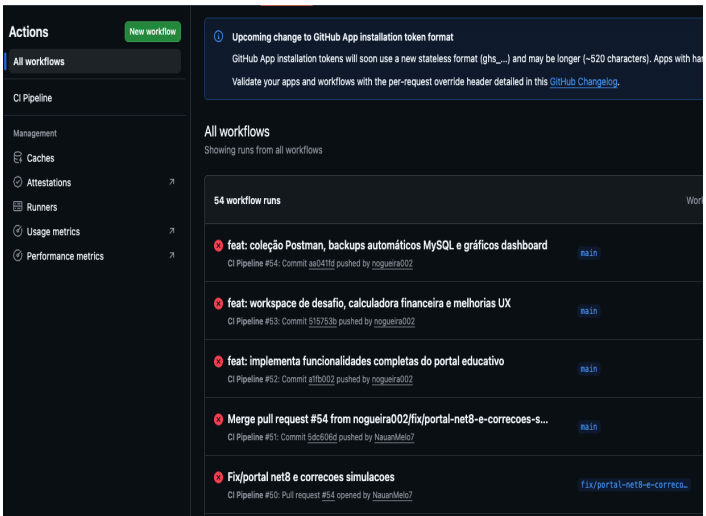


Figura 14 - Pipeline CI/CD Github

Figura 15 - Pipeline CI/CD Github

A pipeline executa três etapas em paralelo:

1. Lint Python - Verifica a qualidade do código Python com flake8, garantindo conformidade com as normas PEP 8.

2. Testes Python - Executa os 27 testes unitários pytest do módulo Django, validando todos os calculators e strategies. A pipeline falha se qualquer teste não passar.

3. Build .NET - Compila o projeto .NET MVC, garantindo que o código compila sem erros antes de qualquer merge.

Esta configuração garante que a qualidade do código é verificada automaticamente, reduzindo o risco de regressões e facilitando a colaboração entre os elementos da equipa.

4.13 API Django — Endpoints

A API Django expõe os seguintes endpoints REST, todos acessíveis via HTTPS através do Nginx no prefixo /api/:

Endpoints Gerais

Método	Endpoint	Descrição
GET	/api/	Informação da API e timestamp
GET	/api/health-check/	Estado do serviço
GET	/api/schema/swagger-ui/	Documentação interativa Swagger
POST	/api/taxa-esforco/	Calcula taxa de esforço com classificação

POST	/api/capitalizacao/	Capitalização composta com TAN personalizada
POST	/api/simular-credito/	Prestação mensal Price e custo total
POST	/api/projecao-poupanca/	Poupança acumulada com inflação
POST	/api/simulate/	Simulação completa com estratégia
GET/ POST	/api/entries/	Lançamentos financeiros por cenário
GET/ POST	/api/objectives/	Objetivos de poupança por cenário
GET/ POST	/api/simulation-history/	Histórico de simulações por cenário

Todos os endpoints retornam exclusivamente JSON, com códigos HTTP adequados — 200 para leituras com sucesso, 201 para criações e 400 para dados inválidos. Os endpoints de cálculo utilizam serializers DRF para validação automática dos dados de entrada, e todos os pedidos são registados com logging estruturado para diagnóstico.

4.14 Descrição das Views e Controllers - Portal .NET MVC

O portal .NET MVC segue a arquitetura Model-View-Controller, com uma separação clara de responsabilidades entre os três componentes.

Controller	**Área**	**Responsabilidade**
AuthController	Transversal	Login, logout, registo, recuperação de password
DashboardController	Aluno	Dashboard com stat cards e gráficos
ScenarioController	Aluno	CRUD de cenários familiares
EntryController	Aluno	CRUD de lançamentos financeiros
ObjectiveController	Aluno	CRUD de objetivos de poupança

SimulationController	Aluno	Formulário de simulação e resultados
ChallengeController	Aluno	Ver e submeter desafios
ClassController	Professor	Criar e gerir turmas
ChallengeManageController	Professor	Criar e gerir desafios
SubmissionController	Professor	Avaliar submissões dos alunos
ProgressController	Professor	Ver progresso por aluno
UserController	Admin	Gerir utilizadores
MonitoringController	Admin	Métricas da plataforma

Fluxo de autenticação e autorização

A autenticação é centralizada no **AuthController**. Após o login, é criada uma sessão por cookie com o perfil do utilizador. Em cada pedido, um middleware de autorização verifica o cookie de sessão e redireciona para a página de login caso o utilizador não esteja autenticado. O acesso às áreas de Professor e Admin é protegido por atributos de autorização que verificam o role do utilizador.

Comunicação com a API Django

Os controllers que necessitam de cálculos financeiros utilizam um serviço DjangoApiService injetado por dependency injection. Este serviço encapsula todas as chamadas HTTP à API Django, tratando erros de comunicação e apresentando mensagens adequadas ao utilizador em caso de falha.

ScenarioController

- DjangoApiService
    - POST /api/simulate/
    - POST /api/taxa-esforco/
    - POST /api/capitalizacao/
    - POST /api/simular-credito/

- POST /api/projecao-poupanca/

Views principais

As views seguem o padrão Razor do .NET MVC, com layouts partilhados por área de utilizador.
O layout principal define a navbar responsiva, sidebar e footer. Cada área, Aluno, Professor, Admin, tem o seu próprio layout parcial com navegação contextual.

As views de simulação implementam JavaScript vanilla para o auto-fill inteligente, ao selecionar um cenário, é feito um pedido AJAX ao controller que agrega os lançamentos do cenário e preenche automaticamente os campos do formulário.

Entity Framework Core e Migrations

O acesso à base de dados dec_educativa é feito através do Entity Framework Core com o padrão Code-First. As migrations são geradas automaticamente e aplicadas no arranque do container, garantindo que o schema está sempre atualizado sem intervenção manual.

4.15 Configuração de Rede e Segurança

Topologia de Rede Docker
Todos os containers comunicam através de uma rede bridge interna chamada **internal**, criada pelo Docker Compose. Esta rede isola completamente os serviços do exterior, apenas o Nginx tem portas expostas ao host.

5. Conclusão

O projeto foi desenvolvido ao longo de três sprints, resultando numa plataforma digital de literacia financeira completa e funcional. O sistema integra um portal educativo em .NET Core MVC com um motor de cálculo financeiro em Django REST e Python, suportados por duas bases de dados MySQL independentes e servidos por Nginx com TLS.

Objetivos Alcançados

Todos os objetivos propostos inicialmente foram atingidos. O sistema implementa autenticação com quatro perfis de acesso, gestão completa de cenários familiares virtuais, simulações financeiras com três estratégias de mercado, sistema pedagógico com turmas e desafios, e infraestrutura conteinerizada com backups automáticos.

5.1. Limitações e Sugestões Futuras

5.1.1 Dificuldades Encontradas

A principal dificuldade foi a reestruturação da arquitetura a meio do Sprint B, resultante da adenda às especificações. A transição de três serviços separados para dois módulos distintos implicou reorganizar o trabalho da equipa e migrar código já desenvolvido.

A gestão de duas bases de dados independentes sem chaves estrangeiras entre elas exigiu cuidado adicional na validação de consistência a nível aplicacional, nomeadamente na verificação da existência de cenários antes de operações financeiras.

5.1.2 Sugestões Futuras

- Implementar notificações automáticas para prazos de entrega de desafios
- Adicionar exportação de relatórios em PDF e Excel
- Permitir taxa de juro personalizada por cenário, independente das estratégias predefinidas
- Implementar simulação Monte Carlo para análise de risco avançada
- Adicionar suporte mobile nativo com aplicação iOS/Android

5.2. Considerações sobre a Inteligência Artificial no Contexto deste Projeto

No desenvolvimento do projeto, a Inteligência Artificial foi utilizada de acordo com o sistema de semáforos definido nas especificações, servindo como ferramenta de apoio e não como substituta do pensamento crítico da equipa.

As utilizações de nível verde, sem justificação, incluíram a geração de comentários e documentação de código, criação do README para GitHub, geração de dados para testes e desenvolvimento de elementos de frontend.

As utilizações de nível amarelo, com justificação e pensamento crítico aplicado, incluíram sugestões de arquitetura para os padrões Strategy e Factory, propostas de queries SQL otimizadas, apoio na configuração do Docker e do servidor, análise de mensagens de erro e debugging, e sugestões de testes unitários que foram depois adaptadas e validadas pela equipa.

As áreas de nível vermelho, proibidas, foram estritamente respeitadas. A modelação e arquitetura de software, os algoritmos base dos calculators financeiros, as regras de negócio e a análise e interpretação dos resultados foram totalmente desenvolvidas exclusivamente pela equipa toda, sem recurso a IA.

Referências:

Microsoft. (2024). *ASP.NET Core MVC documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/mvc/overview

Microsoft. (2024). *Entity Framework Core documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/ef/core/

Django Software Foundation. (2024). *Django REST Framework documentation*. https://www.django-rest-framework.org/

Python Software Foundation. (2024). *Python 3.12 documentation*. https://docs.python.org/3.12/

Docker Inc. (2024). *Docker Compose documentation*. https://docs.docker.com/compose/

Nginx Inc. (2024). *Nginx documentation*. https://nginx.org/en/docs/

MySQL. (2024). *MySQL 8.0 Reference Manual*. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Banco de Portugal. (2024). *Taxas de juro e inflação — dados estatísticos*. https://www.bportugal.pt/estatisticas

pytest. (2024). *pytest documentation*. https://docs.pytest.org/en/stable/

GitHub. (2024). *GitHub Actions documentation*. https://docs.github.com/en/actions

ISO 214. (1976). *Documentation — Abstracts for publications and documentation*. International Organization for Standardization.